📘 picoCTF GDB & Assembly Challenges - Complete Notes

Table of Contents

---

1. Basic Approach

```
┌──────────────────────────────────────┐
│  1. Read assembly TOP TO BOTTOM      │
│  2. Track eax (return value register)│
│  3. IGNORE setup code:               │
│     endbr64, push rbp, mov rbp,rsp   │
│  4. FOCUS on last mov eax before ret │
│  5. Convert hex → decimal for flag   │
└──────────────────────────────────────┘
```

---

2. Common Instructions

Instruction	Meaning	Example
`mov eax, imm`	Move value into eax	`mov eax, 0x86342`
`mov eax, [addr]`	Load from memory	`mov eax, [rbp-0x4]`
`add eax, val`	Add to eax	`add eax, 0x1f5`
`sub eax, val`	Subtract from eax	`sub eax, 0x65`
`imul eax, val`	Multiply eax by value	`imul eax, 0x4`
`imul eax, eax, val`	Multiply eax by constant	`imul eax, eax, 0x3269`
`cmp a, b`	Compare a and b	`cmp eax, 0x2710`

Table 1 – continued

Instruction	Meaning	Example
`jle label`	Jump if Less or Equal	`jle 0x5555...`
`jl label`	Jump if Less	`jl 0x40112c`
`jg label`	Jump if Greater	`jg 0x40112c`
`jmp label`	Always jump	`jmp 0x401136`
`call func`	Call function	`call 0x401106 <func1>`
`ret`	Return (eax holds return value)	—

---

3. Hex to Decimal Conversion

Method: Place Value Expansion

Example: Convert `0x9fe1a`

Position (right→left)	Digit	Decimal	× 16^pos	Result
0	a	10	× 1	10
1	1	1	× 16	16
2	e	14	× 256	3,584
3	f	15	× 4,096	61,440
4	9	9	× 65,536	589,824

Total = 10 + 16 + 3,584 + 61,440 + 589,824 = 654,874

Hex Digit Reference:

```
0-9 = 0-9
a=10, b=11, c=12, d=13, e=14, f=15
```

Powers of 16:

Power	Value
16	1
16^1	16
16^2	256
16^3	4,096
16	65,536
16	1,048,576
16	16,777,216
16	268,435,456

---

4. Challenge Solutions

---

debugger0_a / GDB Baby Step 1

Assembly:

```
<+15>:  mov eax, 0x86342
<+20>:  pop    rbp
<+21>:  ret
```

Analysis:

- Direct move into eax
- Value: `0x86342`

Conversion:

- `0x86342` = `8×65536 + 6×4096 + 3×256 + 4×16 + 2`
- = `524,288 + 24,576 + 768 + 64 + 2`
- = **549,698**

Flag: `picoCTF{549698}`

---

debugger0_b / GDB Baby Step 2

Assembly:

```
<+15>:  mov [rbp-0x4], 0x1e0da    → result = 123,098
<+22>:  mov [rbp-0xc], 0x25f      → limit = 607
<+29>:  mov [rbp-0x8], 0x0        → counter = 0
<+36>:  jmp <main+48>             → go to condition check

LOOP BODY:
<+38>:  mov eax, [rbp-0x8]        → eax = counter
<+41>:  add [rbp-0x4], eax        → result = result + counter
<+44>:  add [rbp-0x8], 0x1        → counter++

CONDITION:
<+48>:  mov eax, [rbp-0x8]        → eax = counter
<+51>:  cmp eax, [rbp-0xc]        → compare with limit
<+54>:  jl <main+38>              → if counter < limit, loop

<+56>:  mov eax, [rbp-0x4]        → eax = result (returned)
```

Analysis:

```
result = 0x1e0da;  // 123,098
limit = 0x25f;     // 607

counter = 0;
```

```
while (counter < limit) {
    result = result + counter;
    counter++;
}
return result;
```

Loop adds: `0 + 1 + 2 + ... + 606`

- Sum = `606 × 607 / 2 = 183,921`

Final: `123,098 + 183,921 = 307,019`

Flag: `picoCTF{307019}`

---

debugger0_c / GDB Baby Step 3

Challenge Description:

> "Examine byte-wise the memory that the constant is loaded in by using the GDB command x/4xb addr. The flag is the four bytes as they are stored in memory."

Assembly:

```
<+15>:  mov DWORD PTR [rbp-0x4], 0x2262c96b
<+22>:  mov eax, [rbp-0x4]
<+26>:  ret
```

Analysis:

- Value `0x2262c96b` stored in memory
- x86 is **little-endian** → bytes stored REVERSED

Split into bytes:

```
0x2262c96b = [22] [62] [c9] [6b]
             MSB →         → LSB
```

In memory (little-endian):

Address	Byte	
[rbp-0x4]	`0x6b` rbp-0x3>	rbp-0x3
	`0xc9` rbp-0x2>	rbp-0x2
	`0x62` rbp-0x1>	rbp-0x1
	`0x22`	

GDB command:

```
x/4xb $rbp-0x4
# Output: 0x6b 0xc9 0x62 0x22
```

Flag format: `picoCTF{0x6bc96222}`

Flag: `picoCTF{0x6bc96222}`

---

debugger0_d / GDB Baby Step 4

Challenge Description:

> "main calls a function that multiplies eax by a constant. The flag is that constant in decimal base."

main Assembly:

```
<+19>:  mov [rbp-0x4], 0x28e        → store 0x28e
<+33>:  mov eax, [rbp-0x4]          → eax = 0x28e
<+36>:  mov edi, eax                → arg1 = 0x28e
<+38>:  call func1                   → call func1(0x28e)
<+43>:  mov [rbp-0x8], eax          → store return value
<+46>:  mov eax, [rbp-0x4]          → eax = 0x28e (this is returned!)
<+50>:  ret
```

func1 Assembly:

```
<+8>:   mov [rbp-0x4], edi          → store argument
<+11>:  mov eax, [rbp-0x4]          → eax = argument
<+14>:  imul eax, eax, 0x3269       → eax = eax × 0x3269
<+21>:  ret
```

Key Insight:

- The challenge asks for the **imul constant**, NOT the return value
- Constant: `0x3269`

Conversion:

- `0x3269` = `3×4096 + 2×256 + 6×16 + 9`
- = `12,288 + 512 + 96 + 9`
- = **12,905**

Flag: `picoCTF{12905}`

---

disassembler-dump0_b

Assembly:

```
<+15>:  mov [rbp-0x4], 0x9fe1a
<+22>:  mov eax, [rbp-0x4]
<+26>:  ret
```

Analysis:

- Direct load: `eax = 0x9fe1a`

Conversion:

- `0x9fe1a` = `9×65536 + 15×4096 + 14×256 + 1×16 + 10`

- = 589,824 + 61,440 + 3,584 + 16 + 10
- = **654,874**

Flag: picoCTF{654874}

---

disassembler-dump0_c

Assembly:

```
<+15>:  mov [rbp-0xc], 0x9fe1a     → val1 = 654,874
<+22>:  mov [rbp-0x8], 0x4          → val2 = 4
<+29>:  mov eax, [rbp-0xc]          → eax = 654,874
<+32>:  imul eax, [rbp-0x8]         → eax = 654,874 × 4
<+36>:  add eax, 0x1f5              → eax = eax + 501
<+44>:  mov eax, [rbp-0x4]          → load final result
<+48>:  ret
```

Analysis:

- 0x9fe1a = 654,874
- 0x4 = 4
- 0x1f5 = 501

Calculation:

- 654,874 × 4 = 2,619,496
- 2,619,496 + 501 = 2,619,997

Flag: picoCTF{2619997}

---

disassembler-dump0_d

Assembly:

```
<+15>:  mov [rbp-0x4], 0x9fe1a     → value = 654,874
<+22>:  cmp [rbp-0x4], 0x2710       → compare with 10,000
<+29>:  jle <main+37>               → jump if ≤
<+31>:  sub [rbp-0x4], 0x65         → subtract 101
<+35>:  jmp <main+41>               → skip add
<+37>:  add [rbp-0x4], 0x65         → add 101 (NOT TAKEN)
<+41>:  mov eax, [rbp-0x4]          → load result
<+48>:  ret
```

Analysis:

- Compare: 654,874 ≤ 10,000? → **FALSE**
- Don't jump → execute sub
- 654,874 - 101 = 654,773

Flag: `picoCTF{654773}`

---

5. GDB Commands Cheat Sheet

Command	Shortcut	What It Does
`gdb filename`	—	Launch GDB with file
`set disassembly-flavor intel`	—	Use Intel syntax
`disassemble main`	`disas main`	Show main assembly
`disassemble func1`	`disas func1`	Show func1 assembly
`break *main+15`	`b *main+15`	Set breakpoint
`run`	`r`	Execute program
`stepi`	`si`	Execute 1 instruction
`continue`	`c`	Continue to next breakpoint
`info registers`	`i r`	Show all registers
`info registers eax`	`i r eax`	Show eax
`print/d $eax`	`p/d $eax`	Print eax decimal
`print/x $eax`	`p/x $eax`	Print eax hex
`x/4xb $rbp-0x4`	—	Examine 4 bytes in hex
`quit`	`q`	Exit GDB

---

6. Loop Detection

Signs of a Loop:

1. **Initialize** variables
2. **Jump forward** to condition check
3. **Loop body** (work + increment)
4. **Condition check**: `cmp` + conditional jump back

Pattern:

```
    mov counter, 0
    jmp CHECK

LOOP:
    add result, counter
    add counter, 1

CHECK:
    cmp counter, limit
    jl LOOP          ; jump back if counter < limit

    mov eax, result
    ret
```

C Equivalent:

```c
counter = 0;
while (counter < limit) {
    result = result + counter;
    counter++;
}
```

---

7. Conditional Jump Logic

Instruction	Condition	Jump If
`je` / `jz`	Equal / Zero	a == b
`jne` / `jnz`	Not Equal / Not Zero	a != b
`jg`	Greater	a > b (signed)
`jge`	Greater or Equal	a >= b (signed)
`jl`	Less	a < b (signed)
`jle`	Less or Equal	a <= b (signed)
`ja`	Above	a > b (unsigned)
`jb`	Below	a < b (unsigned)

Important:

- `cmp a, b` computes `a - b` and sets flags
- `jle` jumps if `a <= b`
- **If condition is FALSE, execution continues to next instruction**

---

8. Little-Endian Memory

Rule:

- **Least Significant Byte** stored at **lowest address**
- **Most Significant Byte** stored at **highest address**

Example: `0x2262c96b`

Byte	Value	Position
MSB	`0x22`	Byte 3
	`0x62`	Byte 2
	`0xc9`	Byte 1
LSB	`0x6b`	Byte 0

In memory (low to high address):

```
[0x6b] [0xc9] [0x62] [0x22]
 ↑low              high↑
```

GDB command:

```
x/4xb $rbp-0x4
# Shows: 0x6b 0xc9 0x62 0x22
```

Flag format: `picoCTF{0x6bc96222}`

---

9. Common Pitfalls

Mistake	Why It' s Wrong	Correct Approach
Forgetting `imul` is multiply	`imul` = **signed multiply**, not "immediate"	Just multiply
Stopping at first `mov eax`	Last `mov eax` before `ret` matters	Read to the end
Ignoring `add` after `imul`	Multiple operations exist	Trace ALL the way to `ret`
Missing `func1` call	Challenge asks for constant in func1	Disassemble func1 too
Wrong flag format	Some want bytes, not decimal	Read challenge description
Not checking little-endian	Bytes stored reversed in memory	Reverse for memory order
`jle` confusion	`jle` jumps if , not if >	Check condition carefully

---

10. Quick Reference Tables

All Solved Flags

Challenge	Type	Flag
`disassembler-dump0_b`	Simple mov	`picoCTF{654874}`
`disassembler-dump0_c`	Math ops	`picoCTF{2619997}`
`disassembler-dump0_d`	Conditional	`picoCTF{654773}`
`debugger0_a`	Simple mov	`picoCTF{549698}`
`debugger0_b`	Loop	`picoCTF{307019}`
`debugger0_c`	Memory bytes	`picoCTF{0x6bc96222}`
`debugger0_d`	Function call	`picoCTF{12905}`

Challenge Types

Type	Pattern	How to Solve
Simple mov	`mov eax, 0x...`	Convert hex to decimal
Math ops	`imul`, `add`, `sub`	Calculate step by step
Conditional	`cmp` + `jle`/`jl`/`jg`	Check condition, follow correct path
Loop	`cmp` + `jl` back to body	Calculate iterations, sum or multiply
Memory bytes	`x/4xb` in description	Read bytes in little-endian order

Table 10 – continued

Type	Pattern	How to Solve
Function call	`call func1`	Disassemble func1, find constant

💡 Pro Tips

1. **Always read the challenge description** — it tells you what to find
2. **Write out the calculation** on paper — don't trust mental math
3. **Use GDB to verify** — `print/d 0x...` confirms your conversion
4. **Check for little-endian** — when memory bytes are involved
5. **Disassemble all functions** — not just main
6. **The last `mov eax` before `ret` is the return value** — unless challenge says otherwise

---

Remember: Assembly is just step-by-step math. Read it like a recipe!